

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	SYED MUHAFIZ A	Reg. No.:	192472282
Section / Batch:		Date:	

Code of Conduct

I SYED MUHAFIZ A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

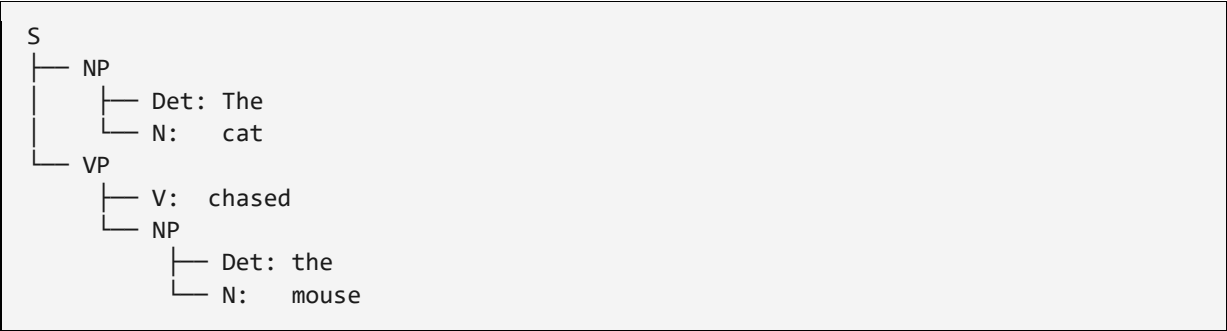
1. CFG Trees vs. Dependency Parsing Structures

Both Context-Free Grammar (CFG) parse trees and dependency structures are ways of representing a sentence's syntax, but they encode fundamentally different information: CFG trees are constituency-based (they group words into nested phrases), while dependency structures are relation-based (they connect individual words directly to one another via labeled, typed grammatical relations).

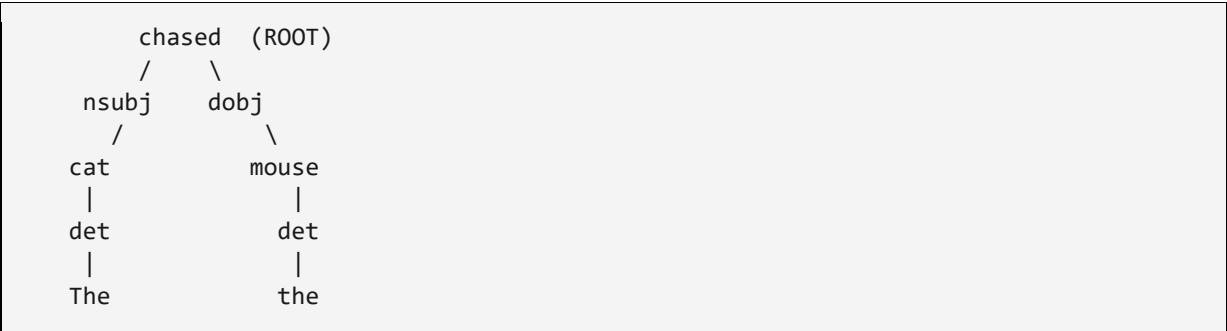
1.1 Worked Example

Sentence: "The cat chased the mouse."

CFG (Constituency) Tree



Dependency Tree



The CFG tree tells us the sentence decomposes into a Noun Phrase ("The cat") and a Verb Phrase ("chased the mouse"), and that the VP further decomposes into a verb and an object NP — but it does not directly label "cat" as the subject or "mouse" as the object; that has to be inferred from tree position. The dependency tree, by contrast, states this directly: cat is attached to chased with the label nsubj (nominal subject) and mouse is attached to chased with the label dobj (direct object) — the grammatical relation is explicit at every edge.

1.2 Structural Comparison

Aspect	CFG (Constituency) Trees	Dependency Structures
Basic unit represented	Phrases/constituents (NP, VP, PP, ...)	Direct word-to-word relations
Shape	Hierarchical, nested, non-terminal-heavy	Flatter graph of labeled head-dependent arcs
Relation labeling	Implicit — inferred from tree position	Explicit — every arc carries a grammatical-role label (nsubj, dobj, amod, ...)
Sensitivity to word order	High — most CFG rules assume a fixed constituent order, harder to adapt to free-word-order languages	Low — relations are defined between words regardless of surface order, making it more portable across languages

Aspect	CFG (Constituency) Trees	Dependency Structures
Ease of extracting “who did what to whom”	Requires tree traversal and rule interpretation	Read directly off the labeled arcs
Typical strength	Full phrase-structure analysis; useful for generation and constituency-based grammar rules	Direct grammatical-relation extraction; useful for agreement checking, information extraction, semantic role labeling

Table 1.1 – CFG constituency trees vs. dependency structures

1.3 Justified Recommendation

Verdict

For the specific goal of capturing relationships between words — which is exactly what grammar checking and syntactic-relation analysis require — dependency parsing is the more effective representation. Its labeled head–dependent arcs make subject–verb, verb–object, and modifier relations directly and unambiguously available without any tree-traversal logic, and its relative independence from strict word order makes it more robust across sentence variations and languages. CFG trees remain valuable where full phrase-structure information is needed (e.g., sentence generation, formal grammar rule-checking, or constituency-based structural ambiguity analysis), but for word-relationship-centric tasks such as grammar checking, dependency parsing is the stronger fit and is the representation adopted by most modern industrial grammar checkers and parsers (e.g., Universal Dependencies-based systems).

2. Top-Down Parsing vs. Earley Parsing for Real-Time Input

A real-time application that must parse user input on the fly — including input that is incomplete (still being typed/spoken) or structurally ambiguous — places very different demands on a parser than an offline batch-processing system. The system currently uses top-down parsing; Earley parsing is the proposed alternative.

2.1 How Each Technique Makes Parsing Decisions

- Top-down parsing starts from the grammar's start symbol and predicts a rule expansion before consuming input tokens; when the prediction fails to match the actual tokens, the parser must backtrack and try the next alternative rule, repeating this predict-and-verify cycle recursively down the grammar.
- Earley parsing builds a chart of parser states incrementally, left to right, one input token at a time; at each position it maintains every partially-matched rule that is still consistent with the tokens seen so far, using a dynamic-programming table so that no sub-derivation is ever discarded and recomputed the way top-down backtracking discards and retries failed predictions.

2.2 Comparative Analysis

Dimension	Top-Down Parsing	Earley Parsing
Decision strategy	Predicts a rule, then checks it against input; retries on mismatch	Incrementally extends a shared chart of all live partial parses
Backtracking	Required whenever a prediction turns out wrong	Not required — all valid alternatives are kept as parallel chart states
Left recursion	Breaks (infinite recursion) unless the grammar is rewritten	Handled natively without grammar transformation
Ambiguous input	Explores one interpretation at a time; costly to recover all parses	Naturally represents multiple simultaneous interpretations in the same chart
Incomplete / partial input	Poorly suited — a partially expanded prediction has no clean way to “pause” and resume	Well suited — the chart at any prefix position already reflects every grammatically consistent continuation so far
Worst-case time complexity	Exponential on ambiguous or deeply backtracking grammars	$O(n^3)$ worst case; close to linear on typical, mildly ambiguous grammars
Suitability for streaming/real-time input	Degrades under ambiguity and noisy/partial input	Designed for exactly this — incremental, chart-based, no wasted recomputation

Table 2.1 – Top-down parsing vs. Earley parsing

2.3 Justified Recommendation

Verdict

Earley parsing is the more suitable technique for a real-time system that must handle incomplete or ambiguous input. Its chart-based dynamic-programming design processes input incrementally and keeps all valid partial derivations alive simultaneously, which directly solves the two problems that plague top-down parsing in this setting: expensive backtracking on failed predictions, and poor support for pausing mid-sentence on partial input. Earley's predictable, generally near-linear runtime on realistic grammars also gives more consistent latency, which matters for a responsiveness-sensitive, real-time user-facing application. Top-down parsing

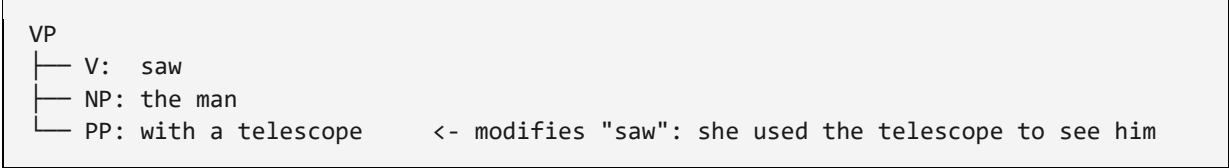
remains attractive only for small, low-ambiguity command grammars where its implementation simplicity outweighs these drawbacks.

3. CFG vs. PCFG vs. Neural Parsing for Ambiguity Resolution

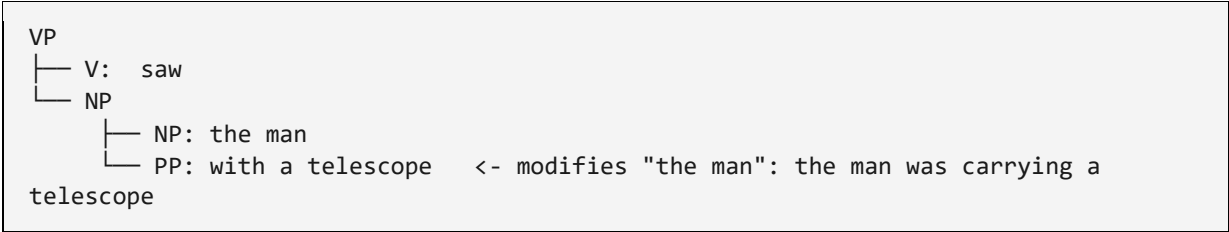
The sentence “She saw the man with a telescope.” is a canonical example of prepositional-phrase attachment ambiguity: “with a telescope” can describe the instrument used for seeing, or it can describe the man himself.

3.1 The Two Readings

Reading 1 — Instrument reading: the PP attaches to the verb phrase



Reading 2 — Modifier reading: the PP attaches to the object noun phrase



3.2 How Each Technique Handles This Ambiguity

- CFG: both trees are equally well-formed under a standard grammar, and plain CFG has no internal mechanism to prefer one over the other — it either returns both trees as an ambiguous parse forest or arbitrarily returns whichever the parsing algorithm builds first, leaving the actual disambiguation decision unresolved.
- PCFG: each production rule (including the PP-attachment rule in both its VP-attachment and NP-attachment forms) carries a learned probability estimated from a treebank; the parser then returns the derivation with the highest overall probability (e.g., via Viterbi/CKY-with-probabilities). This resolves the ambiguity numerically, but a plain (unlexicalized) PCFG bases its choice only on rule frequency, not on the specific words involved — it cannot easily learn that “telescope” specifically favours the instrument reading with a perception verb like “saw” unless the model is lexicalized (i.e., rule probabilities are conditioned on the actual head words, as in the Collins/Charniak-style lexicalized PCFGs).
- Neural parsing: modern neural parsers (e.g., transformer/BiLSTM-based dependency or constituency parsers using contextual word embeddings) learn dense, context-sensitive representations of each word, so the model can directly capture that “telescope” is a strong instrument-of-“see” cue from having encountered many similar sentences during pretraining, without needing hand-specified lexical rule probabilities. This lets neural parsers draw on broad distributional/semantic knowledge, not just local syntactic frequency, when resolving attachment ambiguity.

3.3 Comparative Analysis

Aspect	CFG	PCFG	Neural Parsing
Ambiguity resolution mechanism	None — returns all valid parses equally	Rule-frequency probabilities select the most likely derivation	Learned contextual/lexical-semantic representations

Aspect	CFG	PCFG	Neural Parsing
			select the most likely derivation
Uses lexical/word-specific evidence	No	Only if lexicalized (head-word-conditioned)	Yes, natively, via word/context embeddings
Captures real-world/semantic plausibility	No	Limited	Substantial, via patterns learned from large corpora
Data/training requirement	None (hand-written rules)	Treebank of parsed, frequency-annotated sentences	Large annotated corpora and significant compute for training
Interpretability	High — fully symbolic and traceable	Moderate — probabilities are traceable to rule statistics	Lower — attention/embedding-based decisions are harder to audit
Typical real-world accuracy on attachment ambiguity	Not applicable (no ranking)	Good, but plateaus without lexicalization	State-of-the-art on most benchmark ambiguity-resolution tasks

Table 3.1 – CFG vs. PCFG vs. neural parsing on ambiguity handling

3.4 Justified Recommendation

Verdict

Neural parsing is the most effective approach for real-world applications, because ambiguity resolution of the kind seen in “She saw the man with a telescope” ultimately depends on lexical-semantic and world knowledge (which nouns plausibly serve as instruments for which verbs), and neural models learn exactly this kind of contextual association from large corpora far more effectively than either an unlexicalized PCFG's rule-frequency statistics or plain CFG's purely structural rules. That said, neural parsers are comparatively opaque and resource-intensive; in settings where interpretability, auditability, or low-resource/low-compute constraints matter (e.g., regulated domains or embedded systems), a lexicalized PCFG offers a reasonable, more transparent middle ground, and a hybrid neural-scored, symbolically-constrained pipeline is often the best practical choice for production systems that need both accuracy and explainability.

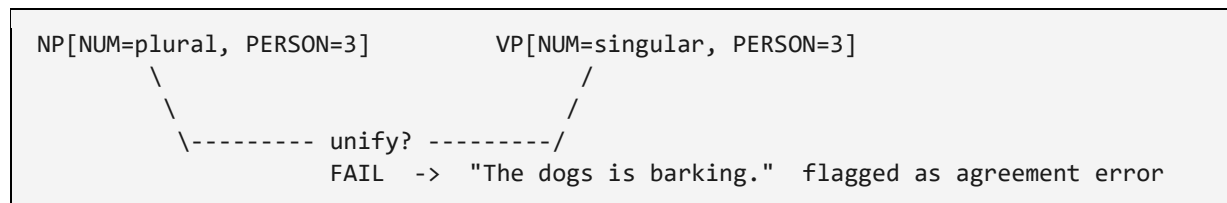
4. Feature Structures vs. Subcategorization Frames

Correct grammar checking requires two distinct kinds of constraint: agreement (do the words match in number, person, gender, tense?) and argument structure (does a verb have the right number and type of dependents — e.g., can it take a direct object, does it require a prepositional complement?). Feature structures and subcategorization frames are the two classical mechanisms used to enforce these constraints.

4.1 What Each Mechanism Represents

Feature Structures

A feature structure is an attribute-value matrix (AVM) attached to a grammar symbol, encoding morphosyntactic properties such as NUMBER, PERSON, GENDER, and TENSE. Two constituents can only combine in a parse if their relevant features unify (are compatible); a mismatch (e.g., a plural subject unifying with a singular-agreement verb) blocks the combination and flags the sentence as ungrammatical.



Subcategorization Frames

A subcategorization frame specifies the argument structure a particular verb (or other predicate) requires — how many arguments, and of what syntactic category — independent of agreement features.

sleep	: SUBCAT < NP[subject] >	(intransitive – no object allowed)
give	: SUBCAT < NP[subject], NP[obj], NP[obj2] >	(ditransitive)
give	: SUBCAT < NP[subject], NP[obj], PP[to] >	(dative-alternation form)
rely	: SUBCAT < NP[subject], PP[on] >	(obligatory PP complement)
"She slept the book." -> violates sleep's SUBCAT frame (extra NP argument not licensed)		
"He relies." -> violates rely's SUBCAT frame (missing required PP[on])		

4.2 Comparative Analysis

Aspect	Feature Structures	Subcategorization Frames
What is checked	Morphosyntactic agreement (number, person, gender, tense)	Verb/predicate argument structure (valency, complement type)
Mechanism	Unification of attribute-value pairs across constituents	Matching a predicate's dependents against a required argument list
Scope of application	Broad — nouns, verbs, determiners, adjectives, across most categories	Narrow — primarily predicates (verbs, and some nouns/adjectives)
Errors caught	"The dogs is barking" (agreement mismatch)	"She slept the book" / "He relies." (wrong or missing arguments)
Extensibility	Naturally extends to encode almost any grammatical property	Purpose-built only for argument/valency constraints

Aspect	Feature Structures	Subcategorization Frames
	as an attribute, including a verb's own SUBCAT list	
Formal grammar tradition	Central to unification-based grammars (HPSG, LFG)	A component typically embedded inside those same unification-based grammars as one more feature

Table 4.1 – Feature structures vs. subcategorization frames

4.3 Justified Recommendation

Verdict

Feature structures provide the better overall support for enforcing grammatical correctness, because they are the more general mechanism — in fact, modern unification-based grammar formalisms such as HPSG represent a verb's subcategorization requirements themselves as a listed feature (e.g., a SUBCAT or ARG-ST attribute) inside the verb's own feature structure. This means feature structures can subsume subcategorization checking while also handling agreement, tense, and other morphosyntactic constraints within a single unification mechanism, whereas subcategorization frames alone only address argument structure and say nothing about agreement. In practice the two are complementary rather than competing — a robust grammar-checking system needs both agreement checking and valency checking — but if the goal is a single, extensible formal mechanism to build the whole grammatical-correctness system on, feature structures are the stronger and more comprehensive foundation.

5. Transition-Based vs. Graph-Based Dependency Parsing

Large-scale NLP systems that must parse dependency structures over big datasets face a direct trade-off between how a parser makes its structural decisions (locally, one step at a time, vs. globally, over the whole sentence at once) and how that choice affects speed and accuracy at scale.

5.1 How Each Method Makes Decisions

- Transition-based parsing treats dependency parsing as a sequence of actions (e.g., SHIFT, LEFT-ARC, RIGHT-ARC in an arc-standard system) applied to a stack-and-buffer configuration; at each step a classifier (traditionally an SVM, now typically a neural network conditioned on local configuration features) predicts the next best action, and the parse tree is built incrementally, left to right, one local decision at a time.
- Graph-based parsing treats parsing as a global optimization problem: it scores every possible head-dependent edge between all word pairs in the sentence (often with a neural scoring function), then finds the highest-scoring valid dependency tree over the whole sentence at once — typically via a maximum spanning tree algorithm (e.g., the Chu-Liu-Edmonds algorithm for general graphs, or Eisner's algorithm for the projective case).

5.2 Comparative Analysis

Dimension	Transition-Based Parsing	Graph-Based Parsing
Decision scope	Local — one action at a time based on the current stack/buffer state	Global — scores all possible edges and finds the best whole-sentence tree
Typical algorithmic complexity	$O(n)$ to $O(n \log n)$ with beam search	$O(n^2)$ (first-order MST) to $O(n^3)$ (Eisner-style projective algorithms); higher for richer factorizations
Error propagation risk	Higher — an early wrong local decision can cascade through the rest of the parse	Lower — the globally optimal tree is chosen with full-sentence information
Handling of long-distance dependencies	Can struggle without careful feature engineering or dynamic-oracle training	Naturally considers long-distance edges since all pairs are scored
Non-projective structures	Requires special transition sets (e.g., swap actions) to handle non-projectivity	Naturally handled by general MST algorithms (e.g., Chu-Liu-Edmonds) without special-casing
Throughput at scale	Very high — near-linear time makes it well suited to large corpora and real-time use	Lower — quadratic/cubic complexity becomes a bottleneck on very large-scale or long-sentence workloads
Typical accuracy (modern neural versions)	Strong, and competitive with graph-based when trained with beam search / dynamic oracles	Often marginally higher on complex or long-distance structures, at higher computational cost

Table 5.1 – Transition-based vs. graph-based dependency parsing

5.3 Justified Recommendation

Verdict

For large-scale NLP systems where processing speed and scalability across big datasets are primary constraints, transition-based parsing is generally more suitable: its near-linear time complexity scales far better to large corpora and high-throughput/real-time pipelines than graph-

based parsing's quadratic-to-cubic complexity. Modern neural transition-based parsers (using beam search and dynamic-oracle training to mitigate the classic error-propagation weakness) have also substantially closed the accuracy gap with graph-based methods, making the speed advantage less costly in practice than it once was. Graph-based parsing remains the better choice when maximum accuracy on long-distance or non-projective dependency structures is the dominant priority and the computational budget allows for it (e.g., smaller, accuracy-critical datasets) — so the right choice ultimately depends on whether the deployment optimizes primarily for scale/throughput (favouring transition-based) or for peak structural accuracy (favouring graph-based).

6. Consolidated Conclusion

Across all five comparisons, a consistent pattern emerges: the more locally-scoped, purely rule-based, or purely structural technique (CFG, top-down parsing, unlexicalized CFG, subcategorization frames alone, or greedy local transition decisions) is easier to implement and reason about, but leaves a real gap — an inability to rank alternatives, adapt to incomplete input, use lexical-semantic evidence, or capture broader morphosyntactic constraints. The more powerful alternative in each case (dependency parsing, Earley parsing, neural parsing, unified feature structures, and — for scale specifically — transition-based parsing) closes that gap by adding either probabilistic ranking, incremental/global optimization, learned contextual representations, or a more general unifying formalism.

The recurring engineering lesson is the same one seen in the earlier Annexures: production NLP systems rarely rely on a single technique in isolation. The strongest real-world designs combine a fast, scalable core (transition-based dependency parsing, Earley's incremental chart) with a statistically or semantically informed disambiguation layer (PCFG probabilities, neural scoring) and a principled formal mechanism for grammatical constraints (feature-structure unification, ideally subsuming subcategorization). Choosing “which is better” is therefore less about picking a single winner and more about matching each technique's strengths — speed, accuracy, interpretability, or robustness to ambiguity and incomplete input — to the specific demands of the target application.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____